UNIT – 4

DYNAMIC PROGRAMMING

Syllabus: Dynamic Programming: All - Pairs Shortest Paths, Single – Source Shortest paths General Weights, 0/1 Knapsack, Reliability Design, Travelling sales person problem

Introduction of Dynamic Programming

Dynamic Programming is the most powerful design technique for solving optimization problems.

Divide & Conquer algorithm partition the problem into disjoint subproblems solve the subproblems recursively and then combine their solution to solve the original problems.

Dynamic Programming is used when the subproblems are not independent, e.g. when they share the same subproblems. In this case, divide and conquer may do more work than necessary, because it solves the same sub problem multiple times.

Dynamic Programming solves each subproblems just once and stores the result in a table so that it can be repeatedly retrieved if needed again.

Dynamic Programming is a **Bottom-up approach-** we solve all possible small problems and then combine to obtain solutions for bigger problems.

Dynamic Programming is a paradigm of algorithm design in which an optimization problem is solved by a combination of achieving sub-problem solutions and appearing to the "**principle of optimality**".

Characteristics of Dynamic Programming:

Dynamic Programming works when a problem has the following features:-

- **Optimal Substructure:** If an optimal solution contains optimal sub solutions then a problem exhibits optimal substructure.
- **Overlapping subproblems:** When a recursive algorithm would visit the same subproblems repeatedly, then a problem has overlapping subproblems.

If a problem has optimal substructure, then we can recursively define an optimal solution. If a problem has overlapping subproblems, then we can improve on a recursive implementation by computing each subproblem only once.

Elements of Dynamic Programming:

There are basically three elements that characterize a dynamic programming algorithm:-

1. **Substructure:** Decompose the given problem into smaller subproblems. Express the solution of the original problem in terms of the solution for smaller problems.
2. **Table Structure:** After solving the sub-problems, store the results to the sub problems in a table. This is done because subproblem solutions are reused many times, and we do not want to repeatedly solve the same problem over and over again.
3. **Bottom-up Computation:** Using table, combine the solution of smaller subproblems to solve larger subproblems and eventually arrives at a solution to complete problem.

Note: Bottom-up means:-

i. Start with smallest subproblems.

ii. Combining their solutions obtain the solution to sub-problems of increasing size.
iii. Until solving at the solution of the original problem.

Development of Dynamic Programming Algorithm:

It can be broken into four steps:

1. Characterize the structure of an optimal solution.
2. Recursively defined the value of the optimal solution. Like Divide and Conquer, divide the problem into two or more optimal parts recursively. This helps to determine what the solution will look like.
3. Compute the value of the optimal solution from the bottom up (starting with the smallest subproblems)
4. Construct the optimal solution for the entire problem form the computed values of smaller subproblems.

Applications of dynamic programming:

1. Single–Source Shortest paths
2. All-pair Shortest path problem
3. 0/1 knapsack problem
4. Reliability design problem
5. Travelling sales person problem

Differences between Divide & Conquer Method and Dynamic Programming:

Divide & Conquer Method	**Dynamic Programming**
1.It deals (involves) three steps at each level of recursion: **Divide** the problem into a number of subproblems. **Conquer** the subproblems by solving them recursively. **Combine** the solution to the subproblems into the solution for original subproblems.	**1.**It involves the sequence of four steps: o Characterize the structure of optimal solutions. o Recursively defines the values of optimal solutions. o Compute the value of optimal solutions in a Bottom-up minimum. o Construct an Optimal Solution from computed information.
2. It is Recursive.	**2.** It is non Recursive.
3. It does more work on subproblems and hence has more time consumption.	**3.** It solves subproblems only once and then stores in the table.
4. It is a top-down approach.	**4.** It is a Bottom-up approach.
5. In this subproblems are independent of each other.	**5.** In this subproblems are interdependent.
6. For example: Merge Sort & Binary Search etc.	**6. For example:** All - Pairs Shortest Paths, Single – Source Shortest paths etc.
7. It is suitable for solving non-optimization problems also.	7. It is suitable for solving optimization problems.

Comparison between dynamic programming approach versus greedy approach

Dynamic programming	Greedy approach
Is useful for solving multistage optimization problems.	Is useful in solving optimization problems.
Can generate multiple sequences of solutions for the given problem.	Generate only one solution sequence.

1. All – pairs shortest paths:

Given a weighted connected graph (undirected or directed), the all-pairs shortest paths problem asks to find the distances—i.e., the lengths of the shortest paths— from each vertex to all other vertices.

It is convenient to record the lengths of shortest paths in an n×n matrix D called the distance matrix: the element d_{ij} in the ith row and the jth column of this matrix indicates the length of the shortest path from the ith vertex to the jth vertex.

Floyd's algorithm is applicable to both undirected and directed weighted graphs provided that they do not contain a cycle of a negative length. (The distance between any two vertices in such a cycle can be made arbitrarily small by repeating the cycle enough times.) The algorithm can be enhanced to find not only the lengths of the shortest paths for all vertex pairs but also the shortest paths themselves.

Floyd's algorithm computes the distance matrix of a weighted graph with n vertices through a series of n×n matrices:

$$D^{(0)}, \ldots, D^{(k-1)}, D^{(k)}, \ldots, D^{(n)} \qquad (4.1)$$

Each of these matrices contains the lengths of shortest paths with certain constraints on the paths considered for the matrix in question. Specifically, the element $d_{ij}^{(k)}$ in the ith row and the jth column of matrix $D^{(k)}$ (i , j =1, 2, . . . , n, k=0, 1, . . . , n) is equal to the length of the shortest path among all paths from the ith vertex to the jth vertex with each intermediate vertex, if any, numbered not higher than k. In particular, the series starts with $D^{(0)}$, which does not allow any intermediate vertices in its paths ; hence , $D^{(0)}$ is simply the weight matrix of the graph. The last matrix in the series, $D^{(k)}$, contains the lengths of the shortest paths among all paths that can use all n vertices as intermediate and hence is nothing other than the distance matrix being sought.

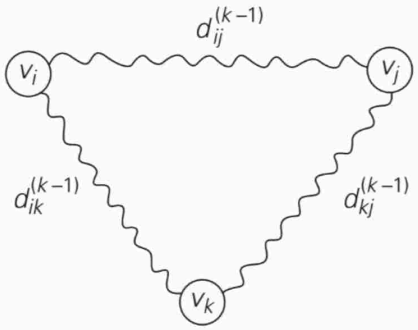
We can compute all the elements of each matrix $D^{(k)}$ from its immediate predecessor $D^{(k-1)}$ in series(4.1). Let $d_{ij}^{(k)}$ be the element in the ith row and the jth column of matrix $D^{(k)}$. This means that $d_{ij}^{(k)}$ is equal to the length of the shortest path among all paths from the ith vertex v_i to the jth vertex v_j with their intermediate vertices numbered not higher than k : v_i, a list of intermediate vertices each numbered not higher than k, v_j.

We can partition all such paths into two disjoint subsets: those that do not use the kth vertex v_k as intermediate and those that do. Since the paths of the first subset have their intermediate vertices numbered not higher than k−1, the shortest of them is, by definition of our matrices, of length $d_{ij}^{(k-1)}$.

If the graph does not contain a cycle of a negative length, we can limit our attention only to the paths in the second subset that use vertex v_k as their intermediate vertex exactly once (because visiting v_k more than once can only increase the path's length). All such paths have the following form :

v_i , vertices numbered $\leq$ k−1 , v_k , vertices numbered $\leq$ k−1, v_j .

In other words, each of the paths is made up of a path from v_i to v_k with each intermediate vertex numbered not higher than k −1 and a path from v_k to v_j with each intermediate vertex numbered not higher than k−1. The situation is depicted symbolically in Figure below:



Since the length of the shortest path from v_i to v_k among the paths that use intermediate vertices numbered not higher than k−1 is equal to $d_{ik}^{(k-1)}$ and the length of the shortest path from v_k to v_j among the paths that use intermediate vertices numbered not higher than k−1 is equal to $d_{kj}^{(k-1)}$, the length of the shortest path among the paths that use the kth vertex is equal to $d_{ik}^{(k-1)} + d_{kj}^{(k-1)}$ .Taking into account the lengths of the shortest paths in both subsets leads to the following recurrence:

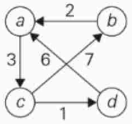
$d_{ij}^{(k)} = \min \{ d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)} \}$ for $k \geq 1$, $d_{ij}^{(0)} = w_{ij}$

To put it another way, the element in row i and column j of the current distance matrix $D^{(k-1)}$ is replaced by the sum of the elements in the same row i and the column k and in the same column j and the row k if and only if the latter sum is smaller than its current value.

```
ALGORITHM    Floyd(W[1..n , 1..n])
  //Implements Floyd's algorithm for the all-pairs shortest-paths problem
  //Input: The weight matrix W of a graph with no negative-length cycle
  //Output: The distance matrix of the shortest paths lengths
  D ←W       //is not necessary if W can be overwritten
  for k←1to n do
   for i ←1to n do
      for j ←1to n do
          D[i, j] ← min { D[i, j] , D[i, k] + D[k, j] }
  return D
```

Time complexity: From the algorithm, the first double for-loop takes $O(n^2)$ time , the nested three for-loops takes $O(n^3)$ time. Thus, the time complexity for All-pairs shortest paths problem (Floyd's algorithm) is $O(n^3)$.

Examples :



$D^{(0)} =$

	a	b	c	d
a	0	∞	3	∞
b	2	0	∞	∞
c	∞	7	0	1
d	6	∞	∞	0

Lengths of the shortest paths with no intermediate vertices ($D^{(0)}$ is simply the weight matrix).

$D^{(1)} =$

	a	b	c	d
a	0	∞	3	∞
b	2	0	**5**	∞
c	∞	7	0	1
d	6	∞	**9**	0

Lengths of the shortest paths with intermediate vertices numbered not higher than 1, i.e., just *a* (note two new shortest paths from *b* to *c* and from *d* to *c*).

$D^{(2)} =$

	a	b	c	d
a	0	∞	3	∞
b	2	0	5	∞
c	**9**	7	0	1
d	6	∞	9	0

Lengths of the shortest paths with intermediate vertices numbered not higher than 2, i.e., a and *b* (note a new shortest path from *c* to *a*).

$D^{(3)} =$

	a	b	c	d
a	0	**10**	3	**4**
b	2	0	5	**6**
c	9	7	0	1
d	6	**16**	9	0

Lengths of the shortest paths with intermediate vertices numbered not higher than 3, i.e., a, *b*, and *c* (note four new shortest paths from *a* to *b*, from a to *d*, from *b* to *d*, and from *d* to *b*).

$D^{(4)} =$

	a	b	c	d
a	0	10	3	4
b	2	0	5	6
c	**7**	7	0	1
d	6	16	9	0

Lengths of the shortest paths with intermediate vertices numbered not higher than 4, i.e., a, *b*, *c*, and *d* (note a new shortest path from *c* to *a*).

Example 4.2 : *Compute all pair shortest path for the following graph.*

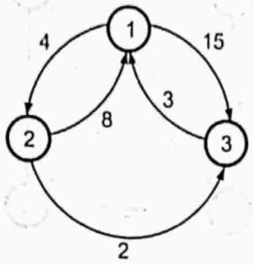


Fig. 4.3 Graph G

Solution :

$$A^0 = \begin{array}{c} \\ 1 \\ 2 \\ 3 \end{array}\begin{bmatrix} 1 & 2 & 3 \\ 0 & 4 & 15 \\ 8 & 0 & 2 \\ 3 & \infty & 0 \end{bmatrix}$$

$$A^1 = \begin{array}{c} \\ 1 \\ 2 \\ 3 \end{array}\begin{bmatrix} 1 & 2 & 3 \\ 0 & 4 & 15 \\ 8 & 0 & 2 \\ 3 & (7) & 0 \end{bmatrix} \rightarrow \min(\infty, (3+4))$$

$$A^2 = \begin{array}{c} \\ 1 \\ 2 \\ 3 \end{array}\begin{bmatrix} 1 & 2 & 3 \\ 0 & 4 & (6) \\ 8 & 0 & 2 \\ 3 & 7 & 0 \end{bmatrix} \rightarrow \min(15, (4+2))$$

$$A^3 = \begin{array}{c} \\ 1 \\ 2 \\ 3 \end{array}\begin{bmatrix} 1 & 2 & 3 \\ 0 & 4 & 6 \\ (5) & 0 & 2 \\ 3 & 7 & 0 \end{bmatrix}$$

A^3 gives shortest distances between any pair of vertices.

2. Single Source Shortest Paths (Bellman–Ford Algorithm):

In bellman-ford algorithm , given a graph “G” and a source vertex “s” in graph, we need to find shortest paths from source vertex to all vertices in the given graph. The graph may contain negative weight edges but no negative cycle . This algorithm works correctly even when some of the edges of the directed graph G may have negative weight. When there are no cycles of negative weight, then we can find out the shortest path between source and destination. It is slower than Dijkstra's Algorithm but more versatile, as it capable of handling some of the negative weight edges.

To solve the problem, first initialize distances from source to all vertices as infinite and distance to source itself as 0. Then apply relaxation rule.

Relaxation Rule :

Relaxation rule must be repeated for |V|-1 times where |V| is the number of vertices in given graph.(i.e. number of iterations=|v|-1)

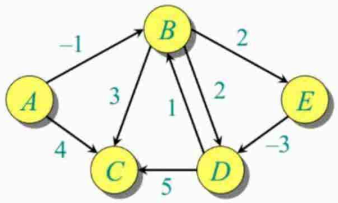
if d[u] + c [u, v] < d[v]

then update d[v] = d[u] + c [u, v]

The **Time complexity** for Single-source shortest path problem (Bellman–Ford)is O(E. |V-1|)i.e. O(E. V)= $O(n^2)$ where E and V are considered as n.
In case of complete graph, the Time complexity is $O(n^3)$

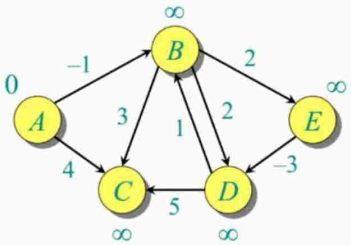
Example :

Graph



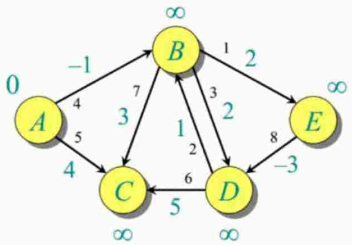
The source vertex is A.

Initialize distances from source "A" to all vertices as infinite and distance to source itself as 0.



Total number of vertices in the graph is 5, so relaxation rule must be applied 4 times.

Let all edges are processed in following order: (B, E), (D, B), (B, D), (A, B), (A, C), (D, C), (B, C), (E, D).

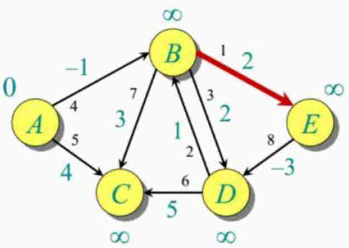


Order of edge relaxation.

Iteration 1:

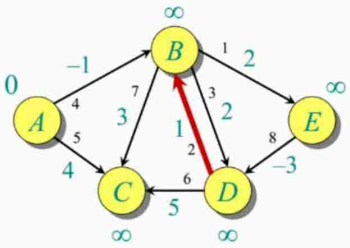
For edge (B, E) ; d[B]=∞ , c[B, E]=2 , d[E]=∞

Since the relaxation condition is not true, we'll not update d[E].



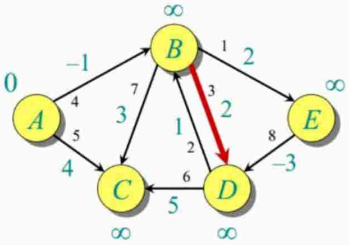
For edge (D, B) ; d[D]=∞ , c[D,B]=1 , d[B]=∞

Since the relaxation condition is not true, we'll not update d[B].



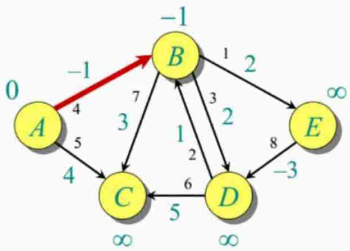
For edge (B, D) ; d[B]=∞ , c[B, D]=2 , d[D]=∞

Since the relaxation condition is not true, we'll not update d[D].



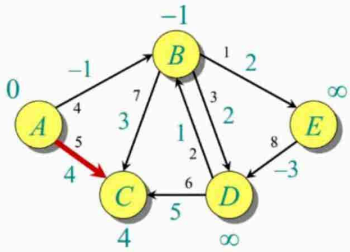
For edge (A,B) ; d[A]=0 , c[A, B]=-1 , d[B]= ∞

Since the relaxation condition is true, we'll update d[B] i.e. (-1)



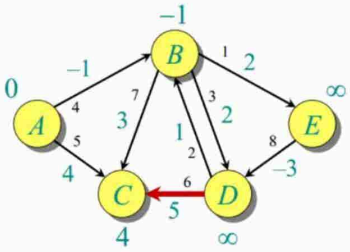
For edge (A,C) ; d[A]=0 , c[A,C]=4 , d[C]=∞

Since the relaxation condition is true, we'll update d[C] i.e. (4)



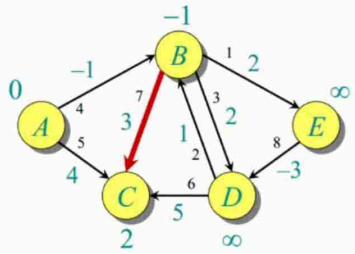
For edge (D,C) ; d[D]=∞ , c[D,C]=5 , d[C]=4

Since the relaxation condition is not true, we'll not update d[C].



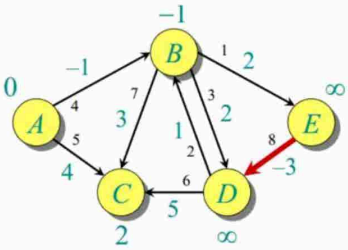
For edge (B, C) ; d[B]=-1 , c[B, C]=3 , d[C]=4

Since the relaxation condition is true, we'll update d[C] i.e. (2)

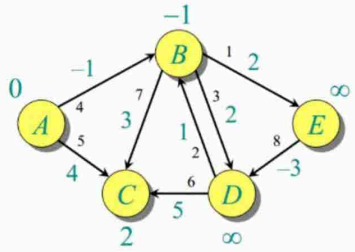


For edge (E,D) ; d[E]=∞ , c[E,D]=-3 , d[D]=∞

Since the relaxation condition is not true, we'll not update d[E].



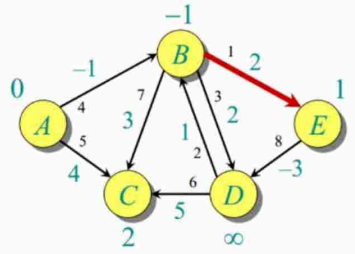
After completion of iteration 1 :



Iteration 2:

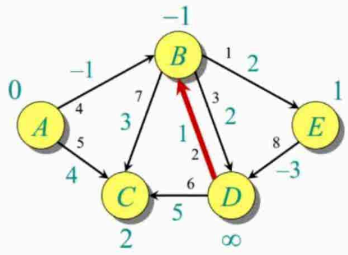
For edge (B, E) ; d[B]=-1 , c[B, E]=2 , d[E]=∞

Since the relaxation condition is true, we'll update d[E] i.e. (1)



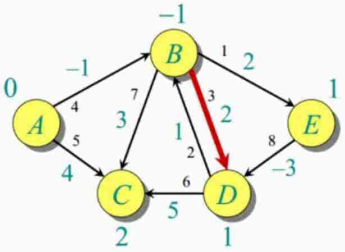
For edge (D, B) ; d[D]=∞ , c[D,B]=1 , d[B]=-1

Since the relaxation condition is not true, we'll not update d[B].



For edge (B, D) ; d[B]=-1 , c[B, D]=2 , d[D]=∞

Since the relaxation condition is true, we'll update d[D] i.e. (1)

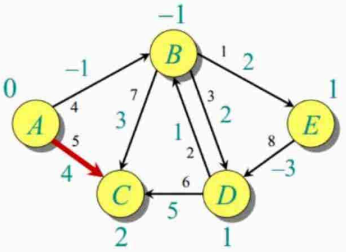


For edge (A,B) ; d[A]=0 , c[A, B]=-1 , d[B]= -1

Since the relaxation condition is not true, we'll not update d[B].

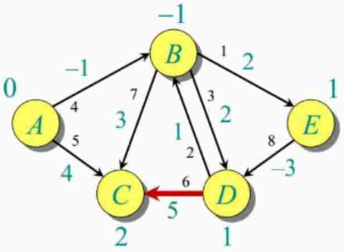
For edge (A,C) ; d[A]=0 , c[A,C]=4 , d[C]=2

Since the relaxation condition is not true, we'll not update d[C].



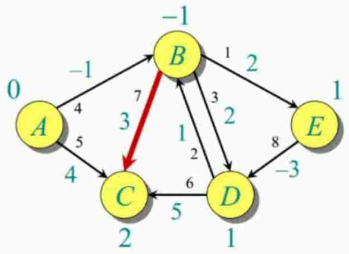
For edge (D,C) ; d[D]=1 , c[D,C]=5 , d[C]=2

Since the relaxation condition is not true, we'll not update d[C].



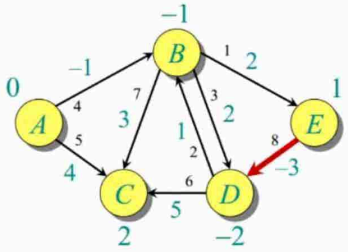
For edge (B, C) ; d[B]=-1 , c[B, C]=3 , d[C]=2

Since the relaxation condition is not true, we'll not update d[C].

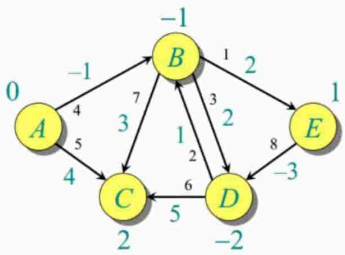


For edge (E,D) ; d[E]=1 , c[E,D]=-3 , d[D]=1

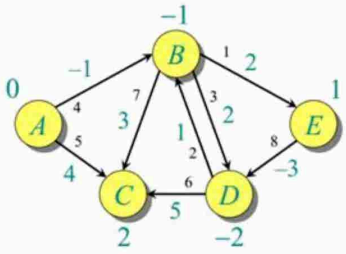
Since the relaxation condition is true, we'll update d[D] i.e. (-2)



After completion of iteration 2:



After completion of iteration 3:



There are no updates of distances from iteration 2 to iteration 3. Therefore the third iteration guarantees to give all shortest paths. The distances are minimized after the third iteration, so fourth iteration don’t update the distances.

Distance of vertex A from source is 0.

Distance of vertex B from source is -1.

Distance of vertex C from source is 2.

Distance of vertex D from source is -2.

Distance of vertex E from source is 1.

Note: The drawback of Bellman-ford algorithm is, it will not work if the graph contains any negative weight cycle.

Comparison between Bellman-ford algorithm and Dijkstra algorithm.

Bellman-ford algorithm	**Dijkstra algorithm**
Deals with single- source shortest-path problem	Deals with single-source shortest-path problem
Allows negative-length edges but does not allow negative cycle.	Does not allow both negative weights and negative cycle.
Is slower than Dijkstra algorithm if more edges are present.	Is faster than Bellman-ford algorithm.

3) 0/1 Knapsack Problem:

For 0/1 knapsack problem in the dynamic programming algorithm:

given n items of known weights w_1 , ...,w_n and values v_1,...,v_n and a knapsack of capacity W, we have to find the most valuable subset of the items that fit into the knapsack. We assume here that all the weights and the knapsack capacity are positive integers; the item values do not have to be integers.

To design a dynamic programming algorithm, we need to derive a recurrence relation that expresses a solution to an instance of the knapsack problem in terms of solutions to its smaller sub instances . Let us consider an instance defined by the first i items, $1 \leq i \leq n$, with weights w_1 , ...,w_n , values v_1,...,v_n , and knapsack capacity j, $1 \leq j \leq W$. Let F(i ,j) be the value of an optimal solution to this instance, i.e., the value of the most valuable subset of the first i items that fit into the knapsack of capacity j. We can divide all the subsets of the first i items that fit the knapsack of capacity j into two categories: those that do not include the ith item and those that do.

Note the following:

1. Among the subsets that do not include the i th item, the value of an optimal subset is, by definition, F(i−1, j).

2. Among the subsets that do include the i th item (hence, $j - w_i \geq 0$),an optimal subset is made up of this item and an optimal subset of the first

i −1 items that fits into the knapsack of capacity $j - w_i$. The value of such an optimal subset is $v_i + F(i-1, j-w_i)$.

Thus, the value of an optimal solution among all feasible subsets of the first i items is the maximum of these two values. Of course, if the i th item does not fit into the knapsack, the value of an optimal subset selected from the first i items is the same as the value of an optimal subset selected from the first i −1 items. These observations lead to the following recurrence:

$$F(i,j) = \begin{cases} \max\{F(i-1,j),\ v_i + F(i-1,j-w_i)\} & \text{if } j - w_i \geq 0, \\ F(-1,j) & \text{if } j - w_i < 0. \end{cases} \quad \text{(Eq 1)}$$

It is convenient to define the initial conditions as follows:

$$F(0,j) = 0 \quad \text{for } j \geq 0 \text{ and}$$

$$F(i,0) = 0 \quad \text{for } i \geq 0. \quad \text{(Eq 2)}$$

Our goal is to find F(n, W), the maximal value of a subset of the n given items that fit into the knapsack of capacity W, and an optimal subset itself.

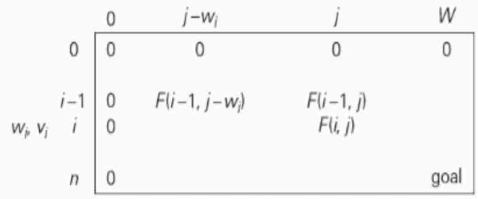


FIGURE 8.4 Table for solving the knapsack problem by dynamic programming.

Figure 8.4 illustrates the values involved in equations 1 and 2. For

i, j > 0, to compute the entry in the i th row and the j th column, F(i ,j), we compute the maximum of the entry in the previous row and the same column and the sum of v_i and the entry in the previous row and w_i columns to the left. The table can be filled either row by row or column by column.

The time needed to find the composition of an optimal solution is O(n).

Example : Let us consider the instance given by the following data:

item	weight	value
1	2	$12
2	1	$10
3	3	$20
4	2	$15

capacity W =5.

The dynamic programming table, filled by applying equations 1 and 2 is shown in Figure 8.5.

		capacity j					
	i	0	1	2	3	4	5
	0	0	0	0	0	0	0
$w_1 = 2,\ v_1 = 12$	1	0	0	12	12	12	12
$w_2 = 1,\ v_2 = 10$	2	0	10	12	22	22	22
$w_3 = 3,\ v_3 = 20$	3	0	10	12	22	30	32
$w_4 = 2,\ v_4 = 15$	4	0	10	15	25	30	**37**

FIGURE 8.5 Example of solving an instance of the knapsack problem by the dynamic programming algorithm.

Thus, the maximal value is F(4, 5)=$37. We can find the composition of an optimal subset by backtracing the computations of this entry in the table. Since F(4, 5) > F(3, 5), item 4 has to be included in an optimal solution along with an optimal subset for filling 5−2=3 remaining units of the knapsack capacity. The value of the latter is F(3, 3). Since

F(3, 3)=F(2, 3), item 3 need not be in an optimal subset. Since

F(2, 3)>F(1, 3), item 2 is a part of an optimal selection, which leaves element F(1, 3−1) to specify its remaining composition. Similarly, since F(1, 2)>F(0, 2), item 1 is the final part of the optimal solution.

The solution is {item 1, item 2, item 4}.

The **Time complexity** for 0/1 Knapsack problem is O (n log n)

4. Reliability Design:

Reliability design problem is based on multiplicative optimization function. We will consider a system in which many devices are connected in series , which is shown below:



n devices D_i , $1 \le i \le n$, connected in series

When devices are connected together then it is a necessity that each device should work properly. The probability that device "i" will work properly is called reliability of that device.

Let r_i be the reliability of device D_i then reliability of entire system is πr_i .

It may happen that even if reliability of individual device is very good but reliability of entire system may not be good. Hence to obtain the good performance from entire system we can duplicate individual devices and can connect them in a series.

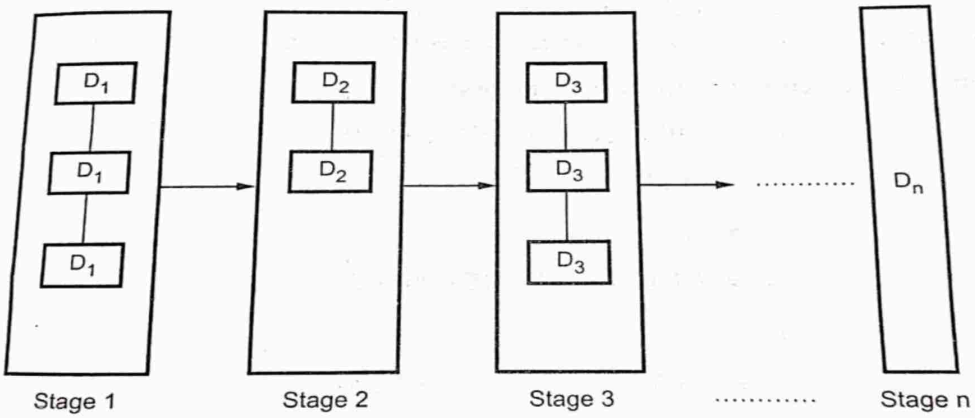


Fig.4.7 Multiple devices connected together

Let m_i be the copies of device D_i then $(1 - r_i)^{mi}$ be the probability that all m_i have a malfunction. Hence the stage reliability is $1 - (1 - r_i)^{mi}$. We will denote reliability of stage i by $\phi_i(m_i)$.

Hence

$$\phi_i(m_i) = 1 - (1 - r_i)^{m_i}$$

In reliability design problem we except to get maximize reliability using device duplication. That means,

Maximize reliability $\prod_{1 \le i \le n} \phi_i(m_i)$ which is subjected to $\sum_{1 \le i \le n} C_i m_i \le C$ where C_i is the cost of each device and C is the maximum allowable cost of the system.

The upper bound u_i on the cost can be determined as :

$$u_i = \left[\left(C + C_i - \sum_{1}^{n} C_j \right) \Big/ C_i \right]$$

The dynamic programming solution S^i consist of tuples of the form (f, x) where $f = f_i(x)$. The $f_i(x)$ can be computed as

$$f_i(x) = \max_{1 \le m_i \le u_i} \left\{ \phi_i(m_i) f_{i-1}(x - C_i m_i) \right\}$$

Initially $f_o(x) = 1$

The dominance rule : $(f_1 x_1)$ dominates $(f_2 x_2)$ iff $f_1 > f_2$ and $x_1 < x_2$. The dominated tuples can be discarded from S^i.

Example :

Example 4.5 : *Design a three stage system with device types D_1, D_2, D_3. The costs are Rs. 30, Rs. 15 and Rs. 20 respectively. The cost of the system is to be no more than Rs. 105. The reliability of each device type is 0.9, 0.8 and 0.5 respectively.*

Solution : Let us compute u_1, u_2 and u_3 first.

$$u_1 = (105 + 30 - (30 + 15 + 20))/30$$

$$= 2.33$$

$$u_1 \approx 2$$

$$u_2 = (105 + 15 - (30 + 15 + 20))/15$$

$$u_2 = 3$$

$$u_3 = (105 + 20 - (30 + 15 + 20))/20$$

$$u_3 = 3$$

Hence $(u_1, u_2, u_3) = (2, 3, 3)$

Initially

$$S^0 = (1, 0)$$

Using S_j we will choose $m_{i,j}$.

then,

$$S_1^1 = \{(0.9, 30)\}$$

$$S_2^1 = \{(0.9, 30)\ (0.99, 60)\} \qquad \because m_i = j = 2$$

In S_2^1 we have $r_i = 0.9$ and $m_i = 2$.

Then $\phi_i(m_i) = 1 - (1 - 0.9)^2$

$$= 0.99$$

and $C_i m_i = 30 \times 2$

$= 60$

Continuing in this fashion we obtain,

$S_1^2 = \{(0.72, 45), (0.792, 75)\}$

$S_2^2 = \{(0.864, 60)\}$

From S_2^2, we have eliminated (0.9504, 90) which can be obtained from (0.99, 60).

For $m_3 = 1$

$S_3^2 = \{(0.8928, 75)\}$

We will combine above computations to obtain

$S^2 = \{(0.72,45), (0.864,60), (0.8928, 75)\}$

Here, (0.792, 75) is dominated by (0.864, 60) hence we will eliminate (0.792, 75).

$S_1^3 = \{(0.36,65), (0.432, 80), (0.4464, 95)\}$

$S_2^3 = \{(0.54,85)\ (0.648, 100)\}$

$S_3^3 = \{(0.63, 105)\}$

We will combine above sets to obtain S^3.

$S^3 = \{(0.36, 65), (0.432, 80), (0.54, 85), (0.648, 100)\}$

Hence the best reliability design will be with reliability 0.648 and Cost 100. For obtaining solution if we trace back then $m_1 = 1$, $m_2 = 2$ and $m_3 = 2$.

5. Travelling sales person problem:

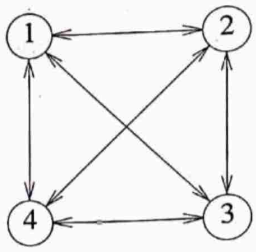
Given a set of cities and distance between every pair of cities, the problem is to find the shortest possible route that visits every city exactly once and returns to the starting point.

Cost function:

$$g(i, S) = \min_{j \in S} \{c_{ij} + g(j, S - \{j\})\} \qquad (5.21)$$

Example 1:

Example 5.26 Consider the directed graph of Figure 5.21(a). The edge lengths are given by matrix c of Figure 5.21(b).



(a)

$$\begin{bmatrix} 0 & 10 & 15 & 20 \\ 5 & 0 & 9 & 10 \\ 6 & 13 & 0 & 12 \\ 8 & 8 & 9 & 0 \end{bmatrix}$$

(b)

Figure 5.21 Directed graph and edge length matrix c

Solution:

Thus $g(2,\phi) = c_{21} = 5, g(3,\phi) = c_{31} = 6$, and $g(4,\phi) = c_{41} = 8$. Using (5.21), we obtain

$$\begin{array}{llll} g(2,\{3\}) = c_{23} + g(3,\phi) = 15 & & g(2,\{4\}) = 18 \\ g(3,\{2\}) = 18 & & g(3,\{4\}) = 20 \\ g(4,\{2\}) = 13 & & g(4,\{3\}) = 15 \end{array}$$

Next, we compute $g(i,S)$ with $|S| = 2$, $i \neq 1$, $1 \notin S$ and $i \notin S$.

$$\begin{array}{lll} g(2,\{3,4\}) = \min\{c_{23} + g(3,\{4\}), c_{24} + g(4,\{3\})\} = 25 \\ g(3,\{2,4\}) = \min\{c_{32} + g(2,\{4\}), c_{34} + g(4,\{2\})\} = 25 \\ g(4,\{2,3\}) = \min\{c_{42} + g(2,\{3\}), c_{43} + g(3,\{2\})\} = 23 \end{array}$$

$$\begin{aligned} g(1,\{2,3,4\}) &= \min\{c_{12} + g(2,\{3,4\}), c_{13} + g(3,\{2,4\}), c_{14} + g(4,\{2,3\})\} \\ &= \min\{35, 40, 43\} \\ &= 35 \end{aligned}$$

An optimal tour of the graph of Figure 5.21(a) has length 35. A tour of this length can be constructed if we retain with each $g(i,S)$ the value of j that minimizes the right-hand side of (5.21). Let $J(i,S)$ be this value. Then, $J(1,\{2,3,4\}) = 2$. Thus the tour starts from 1 and goes to 2. The remaining tour can be obtained from $g(2, \{3, 4\})$. So $J(2, \{3, 4\}) = 4$. Thus the next edge is $\langle 2,4\rangle$. The remaining tour is for $g(4, \{3\})$. So $J(4, \{3\}) = 3$. The optimal tour is 1, 2, 4, 3, 1. □

Example 2:

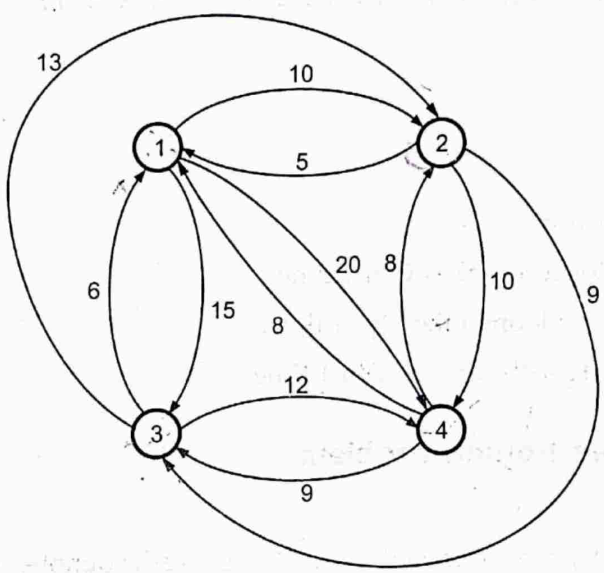


Fig. 4.5

The distance matrix can be given by,

from ↓ / to →	1	2	3	4
1	0	10	15	20
2	5	0	9	10
3	6	13	0	12
4	8	8	9	0

Solution : First we will select any arbitrary vertex say select 1.

Now process for intermediate sets with increasing size.

Step 1 : Let S = ϕ then,

$$\text{Cost } (2, \phi, 1) = d(2,1) = 5$$

$$\text{Cost } (3, \phi, 1) = d(3,1) = 6$$

$$\text{Cost } (4, \phi, 1) = d(4,1) = 8$$

That means we have obtained dist (2,1), dist (3,1) and dist (4,1).

Step 2 :

$$\text{Candidate (S)} = 1$$

Apply the formula,

$$\text{Cost } (i, S) = \min \left\{ d[i,j] + \text{Cost } (j, S - \{j\}) \right\}$$

Hence from vertex 2 to 1, vertex 3 to 1 and vertex 4 to 1 by considering intermediate path lengths we will calculate total optimum cost.

$$\text{Cost } (2, \{3\}, 1) = d(2,3) + \text{Cost } (3, \phi, 1)$$

$$= 9 + 6$$

$$\text{Cost } (2, \{3\}, 1) = 15$$

$$\text{Cost } (2, \{4\}, 1) = d(2,4) + \text{Cost } (4, \phi, 1)$$

$$= 10 + 8$$

$$\text{Cost } (2, \{4\}, 1) = 18$$

$$\text{Cost } (3, \{2\}, 1) = d(3,2) + \text{Cost } (2, \phi, 1)$$

$$= 13 + 5$$

$$\text{Cost } (3, \{2\}, 1) = 18$$

$$\text{Cost } (3, \{4\}, 1) = d(3,4) + \text{Cost } (4, \phi, 1)$$

$$= 12 + 8$$

$$\text{Cost } (3, \{4\}, 1) = 20$$

$$\text{Cost } (4, \{2\}, 1) = d(4,2) + \text{Cost } (2, \phi, 1)$$

$$= 8 + 5$$

$$\text{Cost } (4, \{2\}, 1) = 13$$

$$\text{Cost } (4, \{3\}, 1) = d(4,3) + \text{Cost } (3, \phi, 1)$$

$$= 9 + 6$$

$$\text{Cost}(4, \{3\}, 1) = 15$$

Step 3 : Consider candidate (S) = 2

$$\text{Cost}(2, \{3,4\}, 1) = \min\{[d(2,3) + \text{Cost}(3,\{4\}, 1)], [d(2,4) + \text{Cost}(4,\{3\},1)]\}$$

$$= \min\{[9 + 20], [10+15]\}$$

$$\text{Cost}(2, \{3,4\}, 1) = 25$$

$$\text{Cost}(3, \{2,4\}, 1) = \min\{[d(3,2) + \text{Cost}(2,\{4\}, 1)], [d(3,4) + \text{Cost}(4,\{2\},1)]\}$$

$$= \min\{[13+18], [12+13]\}$$

$$\text{Cost}(3, \{2,4\}, 1) = 25$$

$$\text{Cost}(4, \{2,3\}, 1) = \min\{[d(4,2) + \text{Cost}(2,\{3\}, 1)], [d(4,3) + \text{Cost}(3,\{2\},1)]\}$$

$$= \min\{[8 + 15], [9 + 18]\}$$

$$\text{Cost}(4, \{2,3\}, 1) = 23$$

Step 4 : Consider candidate (S) = 3. i.e. Cost (1,{2,3,4}) but as we have chosen vertex 1 initially the cycle should be completed i.e. starting and ending vertex should be 1.

∴ We will compute,

$$\text{Cost}(1, \{2,3,4\},1) = \min\begin{Bmatrix}[d(1,2) + \text{Cost}(2,\{3,4\},1)], [d(1,3) + \text{Cost}(3,\{2,4\},1)] \\ [d(1,4) + \text{Cost}(4,\{2,3\},1)]\end{Bmatrix}$$

$$= \min([10 + 25], [15 + 25], [20 + 23])$$

$$= 35$$

Thus the optimal tour is of path length 35.

Consider step 4 now, from vertex 1 we obtain the optimum path as d(1,2). Hence select vertex 2. Now consider step 3, in which from vertex 2 we obtain optimum cost from d(2,4). Hence select vertex 4. Now in step 2 we get remaining vertex 3 as d(4,3) is optimum. Hence optimal tour is 1, 2, 4, 3, 1.